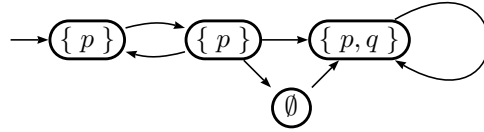


Exercises PV : CTL Model Checking

Wishnu Prasetya

October 31, 2018

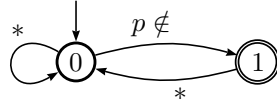
1. Consider the following Kripke structure:



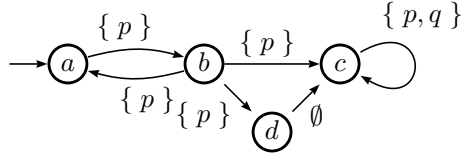
- (a) Do LTL model checking to verify the LTL property $\Diamond\Box p$.

Answer:

First we construct a Buchi automaton representing $\neg\Diamond\Box p = \Box\Diamond\neg p$:

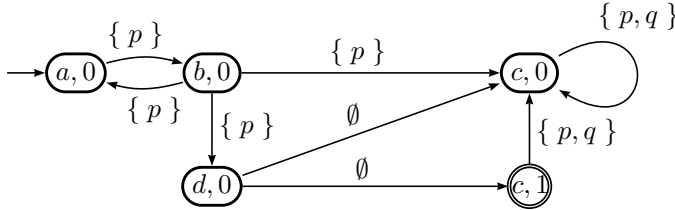


We transform the given Kripke structure (modelling the program to verify) to a Buchi automaton, because it would be easier as we later intersect it with the above specification:



We furthermore label the states with a, b, c, d .

We can now construct a Buchi automaton representing the intersection of the above two Buchis (belonging to the program and the negation of our original specification):



There is only one accepting state, namely $(c, 1)$.

The given specification is violated if we can find a sentence accepted by the above Buchi automaton. This is a infinite sentence which can be produced by an infinite execution that pass through the accepting state above infinitely many times.

This is only the case if the accepting state is reachable (from the initial state), and if it is also part of a cycle. This can be detected e.g. by a nested DFS algorithm as used by SPIN.

In our case, the accepting state is not a part of any cycle. So, the automaton can't produce a counter example. Therefore, it satisfies $\Diamond\Box p$.

- (b) Can the above property be expressed in CTL? How about in CTL*?

Answer: For the above program, the LTL property $\Diamond \Box p$ can't be expressed in CTL. Note that the CTL property **AFAG** p does not hold on the above program.

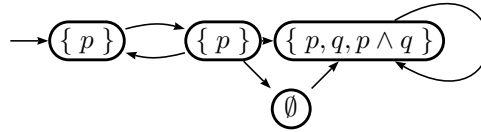
We can however express the property in CTL* as: **AFG** p .

- (c) Do CTL model checking to verify **EF**($p \wedge q$).

Answer: The algorithm works by systematically labelling the states of our program with formulas known to hold on the corresponding states.

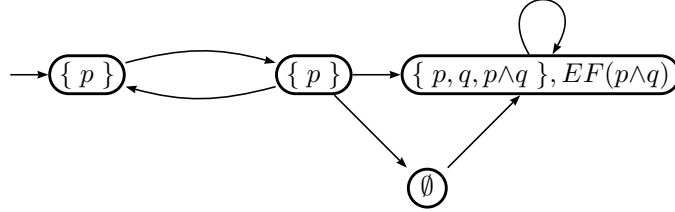
We first label the states with the atomic propositions that occur in our formula. However, we don't have to do anything actually, because the labeling with the atomic propositions are already given in the Kripke itself.

We now continue with the labeling with the subformula $p \wedge q$:

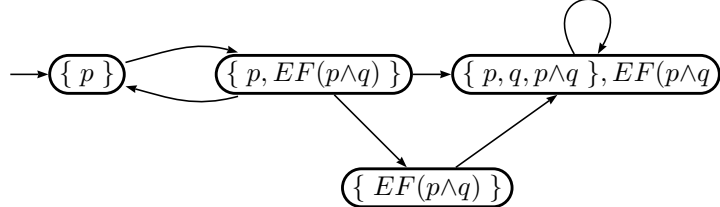


Then the labeling with the formula **EF**($p \wedge q$) itself. First note that the property can also be written as **E**[**true** **U** $p \wedge q$]. The labeling of such a property is done iteratively.

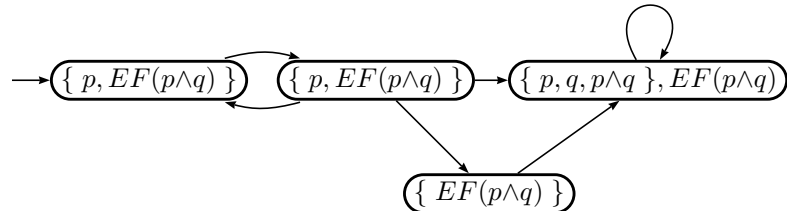
Iteration 1. Those states labelled with $p \wedge q$ obviously satisfy $EF(p \wedge q)$, so we add the latter:



Iteration 2. Next, all states s that has an outgoing transition into a state labelled with $EF(p \wedge q)$ will therefore satisfy $EF(p \wedge q)$; so we add it as labels:



Iteration 3. Applying the same step as in the previous iteration get us to this:



After this we can't label any more states with $EF(p \wedge q)$ (in this case simply because all states have been labelled with this formula). So we stop the iteration.

Note such an iteration will terminate. Each iteration either does not label any new state, in which case we stop, or it labels new states. The latter cannot go on forever because we have a finite number of states.

Now, since the initial state is also labelled with $EF(p \wedge q)$ the formula is therefore holds on this state; and thus the program satisfies it.

(d) Ok, now try these properties:

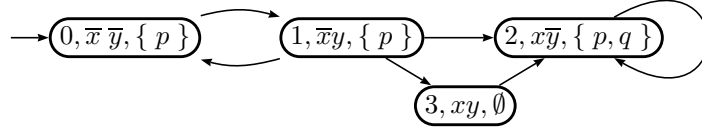
- **EF** $\neg p$
- **AG** p
- **E** $[p \text{ U } (\text{AG } p)]$
- **A** $[p \text{ U } (\text{AG } p)]$
- **AFAG** p

2. Consider again the Kripke structure in No. 2.

(a) How would you describe it if you are to express with a Boolean formula?

Answer:

The four states can be encoded by two boolean variables x, y . Let us first number the states and label them with their encoding. This is just so that we can latter refer to them. We will write $\bar{x}$ to mean $\neg x$.



We can now encode the arrows in this automaton with a boolean formula; each arrow can be encoded by one DNF clause (there are six arrows, notice that below we also have 6 clauses):

$$\begin{aligned}
 R(x, y, x', y') \\
 = \\
 \bar{x} \bar{y} \bar{x}' y' \vee \bar{x} y \bar{x}' \bar{y}' \vee \bar{x} y x' \bar{y}' \vee \bar{x} y x' y' \vee x y x' \bar{y}' \vee x \bar{y} x' \bar{y}'
 \end{aligned}$$

You'll of course get a formula with as many clauses as your arrows in the original automaton. However, we can come up with a more compact formula by expressing the automaton in a more declarative, rule-based, way. E.g.:

- i. from 0, you may go to 1.
- ii. from 1, you don't go back to 1.
- iii. from 2 or 3, you may go to 2.

which can be directly expressed with boolean formulas, that now looks simpler:

$$\begin{aligned}
 R(x, y, x', y') \\
 = \\
 \bar{x} \bar{y} \bar{x}' y' \vee \bar{x} y \bar{x}' \bar{y}' \vee x x' \bar{y}'
 \end{aligned}$$

The sets of states satisfying p respectively q are encoded by the formula:

$$\begin{aligned}
 W_p &= \bar{x} \bar{y} \\
 W_q &= x \bar{y}
 \end{aligned}$$

(b) Do the model checking of the formula **EF** $(p \wedge q)$ on the symbolic representation of your Kripke.

Answer:

CTL symbolic model checking proceeds as CTL explicit state model checking, except that we use boolean formulas. When we label states with a formula, we basically compute the set of states satisfying that formula. We can express set of state with a boolean formula.

For example, in the next stage of labeling we would label all the states satisfying $p \wedge q$. These states are simply those states in the conjunction of W_p and W_q . So:

$$W_{p \wedge q} = W_p \wedge W_q = \overline{x}y x \overline{y} = x \overline{y}$$

Now we proceed with the labelling with $\mathbf{EF}(p \wedge q)$. Remember we do this in iterations, where each iterations basically compute an approximation of the set of states satisfying $\mathbf{EF}(p \wedge q)$.

The first approximation K_1 is simply $W_{p \wedge q}$. The next approximation is:

$$\begin{aligned} K_2 &= \\ &(\exists x', y'. R(x, y, x', y') \wedge K_1[x', y'/x, y]) \vee K_1 \\ &= \\ &(\exists x', y'. R(x, y, x', y') \wedge x' \overline{y'}) \vee K_1 \\ &= \\ &(\exists x', y'. (\overline{x} \overline{y} \overline{x'} y' \vee \overline{x} y \overline{x'} \overline{y'} \vee x x' \overline{y'}) \wedge x' \overline{y'}) \vee K_1 \end{aligned}$$

You will probably wonder if the above complicated formula really represent all the states labelled in the second iteration. It is indeed difficult to see this directly; but just so that you can convince yourself, let's simplify the above formula:

$$\begin{aligned} &\text{last formula above} \\ &= \\ &\overline{x} y \vee x \vee K_1 \\ &= \\ &\overline{x} y \vee x \vee x \overline{y} \\ &= \\ &\overline{x} y \vee x \end{aligned}$$

The above formula encodes this set of states: $\{1, 2, 3\}$. If you recall the labelling in the explicit state procedure, this is indeed the set of states we label with $\mathbf{EF}(p \wedge q)$ in Iteration-2.

The next approximation K_3 can be obtained in the same way:

$$\begin{aligned} K_3 &= \\ &(\exists x', y'. R(x, y, x', y') \wedge K_2[x', y'/x, y]) \vee K_2 \\ &= \\ &(\exists x', y'. R(x, y, x', y') \wedge (\overline{x'} y' \vee x')) \vee \overline{x} y \vee x \end{aligned}$$

At each iteration we should check if $K_{i+1} \leftrightarrow K_i$; if so we stop. I'm not going to write out K_3 above. I leave it to you to check yourself that we can stop after K_4 , because $K_4 \leftrightarrow K_3$. Hence:

$$W_{\mathbf{EF}(p \wedge q)} = K_3$$

Next we need to check if initial state is labelled by $\mathbf{EF}(p \wedge q)$. In terms of symbolic model checking this means checking either of this:

- i. Valuation $x=false$ and $y=false$ makes $W_{\mathbf{EF}(p \wedge q)}$ true.
- ii. $\overline{x} \overline{y} \rightarrow W_{\mathbf{EF}(p \wedge q)}$ is valid.